1 Exercise 5: Timer/Counter and ADC (Microchip Studio) (MEGA)

1.1 Useful References

- ATmega2560 Datasheet
- AVR LIBC manual

1.2 Overview

In this task, you are going to build an analog to digital conversion (ADC) that is triggered by 8-bit timer/counter using MEGA microcontroller. The result of ADC will be printed on the LCD screen. Analog signal is obtained from the thermistor.

1.3 Building Circuit

Build the circuit using these components:

- Arduino Mega (Mega 2560)
- LCD display
- Thermistor
- Potentiometer
- Resistor 10 kOhm
- Resistor 220 Ohm
- Wires

The schematic of the circuit is as follows.

- Connect LCD D4 – D7 pins to MEGA pins 3 – 6
- Connect LCD RS, RW and E pins to MEGA pins 9 - 11
- LCD V0 pin is connected to the middle leg of potentiometer. 2 other legs of potentiometer should be connected to power and ground (order does not matter).
- LCD VDD is connected to 5V bar
- LCD VSS and K are connected to ground
- LCD A pin is connected to 5V bus via 220 Ohm current limiting resistor
- Connect one leg of the thermistor to a 10k Ohm resistor which connects to ground. This thermistor leg should be connected to the analog pin A3 in the Mega. And the other leg is connected to power.
- Connect any of 5V pins of Mega board to your circuit's 5V bar.
- Connect any ground pin of Mega board to your circuit's ground bar.

1.4 Included Code

Create main.c file in your project. Include the following provided headers and libraries in your project and add the required `#include` directives to the main.c unit:

- mcu.h for microcontroller-related defines
- `bit_ops.h` for bit manipulation in registers
- uart.h and uart.c for serial port driver implemented in Exercise 1
- lcd.c and lcd.h for LCD driver

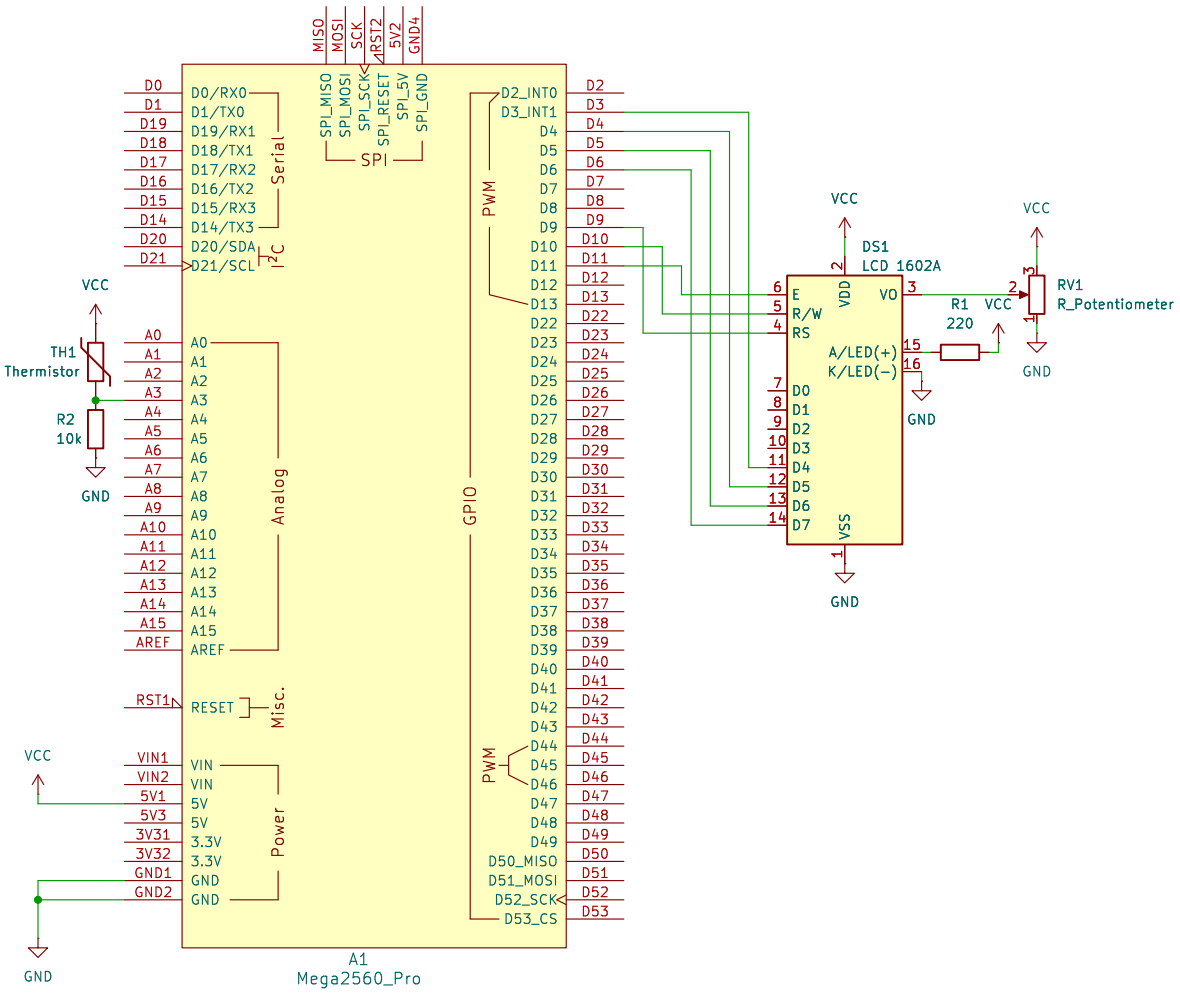

Figure 1: Circuit schematic.

1.5 New bitops

Edit `bit_ops.h` to add new macro that will allow writing of multibit fields.

```
/// Set bits in provided variable in place. Bit-wise OR between the current value and the mask
#define SET_BITS(val, bit, mask) (val |= (((__typeof__(val))mask) << bit))
```

Here mask is the value passed to the field.

1.6 Critical Sections

When interrupts are used in microcontroller the assumptions of sequential execution of your code no longer hold. At this point issues like race conditions and unexpected execution order of commands can produce unexpected and often hard to diagnose bugs. Critical section is a common pattern to recover some control over code execution in embedded systems. The idea is that when you enter the critical section you globally disable handling of all or most interrupts which allows you to think of code execute dwithin the critical section as atomic and sequential. After implementing the critical code the critical section is exited and handling of interrupts is restored. It is also often necessary to execute code that contains a critical section from within another critical section. For that purpose a nesting counter is used that counts up on every enter and down before every exit with interrupts returning to enabled status only when the nesting counter reaches 0.

Create and add to project `critical.h` Implement header-only helpers for critical section handling by using the following declarations in

```
#ifndef CRITICAL_H
#define CRITICAL_H

#include <avr/interrupt.h>
#include <stdio.h>

/// This global keeps track of nested critical sections.
extern uint32_t critical_nesting;

/// Enter critical code section which prevents interrupts until it is exited
static void enter_critical(void) {
   // First disable the interrupts with cli()
   ...
   // Then increment nesting counter to only enable interrupts back when we exit all
   // nested levels
   ...
}

/// Exit critical code section
static void exit_critical(void) {
   // If we are not in critical section log warning as this should not happen.
   ...
   // Otherwise decrement the nesting.
   // If we reached 0 it is time to enable interrupts again with sei()
   ...
}

#endif
```

Then create and add to project a mini module `critical.c` that will store the `critical_nesting` global.

```
#include <stdint.h>

/// This global keeps track of nested critical sections.
uint32_t critical_nesting = 0;
```

1.7 Timer 0 driver

Create and add to project `timer0.h` Add to it the following definitions

```
#ifndef TIMER0_H
#define TIMER0_H

#include <stdint.h>

/// Set up the 8-bit timer/counter0.
/// This function provide precise intervals for triggering
/// ADC conversions and internal timing without busy waiting
void setup_timer0(void);

/// Returns a copy of current wall timer value. Uses critical sections to ensure atomicity.
uint32_t get_time(void);
#endif //TIMER0_H
```

Then create and add to project `timer0.c` and copy the definitions (without the guard section) there. Also add the following includes

```
#include "timer0.h"
#include "mcu.h"
#include <avr/interrupt.h>
#include <avr/io.h>

#include "critical.h"
#include "bit_ops.h"
```

Add and fill (replace exclamation marks) the following declarations using the datasheet

```
/// Datasheet p. 126
#define TCCR0A_WGM_MODE_OFFSET (0)
#define TCCR0A_WGM_MODE_MASK (0b11)

/// Datasheet p. 128 Table 16-8
#define WGM_MODE_CLEAR_ON_COMPARE_MATCH !!!

/// Datasheet p. 129
#define TCCR0B_CLOCK_SELECT_OFFSET (0)
#define TCCRB0_CLOCL_SELECT_MASK (0b111)
#define TCCR0B_WGM_MODE_OFFSET !!!
#define TCCR0B_WGM_MODE_MASK !!!

/// Datasheet p. 130 Table 16-9
#define CLOCK_SELECT_CLKIO_DIV_1024 !!!

/// Datasheet p. 131
#define TIMSK_TIFR_TIMER_OVERFLOW_INTERRUPT_OFFSET !!!
#define TIMSK_TIFR_COMPARE_A_MATCH_INTERRUPT_OFFSET !!!

/// Human-readable registers
#define CONTROL_REGISTER_A (TCCR0A)
#define CONTROL_REGISTER_B (TCCR0B)
#define TIMER_COUNT (TCNT0)
#define OUTPUT_COMPARE_A (OCR0A)
#define OUTPUT_COMPARE_B (OCR0B)
#define INTERRUPT_MASK (TIMSK0)
#define INTERRUPT_FLAGS (TIFR0)

/// Compute timer trigger period
```

```
#define MILLISECONDS_PER_SECOND (1000)
#define CONFIGURED_PRESCALER (1024)
#define OCRA_PERIOD_10MS ((F_CPU * 10) / (MILLISECONDS_PER_SECOND) / (CONFIGURED_PRESCALER))
```

Implement a module-global (static) variable `uint32_t timer_ticks` that will store programs "wall" time with 10 ms resolution and the timer0 interrupt service routine (ISR) that will increment that global every time the timer counts to its compare match value.

```
static uint32_t timer_ticks = 0;

ISR (TIMER0_COMPA_vect)
{
    timer_ticks++;
}
```

Implement module's public interface `get_time`that will enter critical section, copy `timer_ticks` valu to a local variable, exit critical section and return the result. This function will serve as atomic getter for the time value. Note that not using a critical secion here might result in a race condition as a 32-bit integer increment and copy will translate in 4 separate add or mov istructions each in 8-bit AVR assembly. A timer can fire at any time with respect to the call of this getter so it might end up returning 1 to 3 bytes of old value composed with 1 to 3 bytes of the new value causing rare (sometimes extemely rare) glitches in the calling code. Always consider the access to variables shared between an ISR and the rest of your code as requiring a critical section.

Then implement a reset module function that sets all timer registers to their initial value (0) and a helper that will split waveform generation mode (WGM) value into bits located in two registers.

```
/// Set default value in all timer registers.
static void reset_timer0_module(void) {
    CONTROL_REGISTER_A = 0;
    CONTROL_REGISTER_B = 0;
    TIMER_COUNT = 0;
    OUTPUT_COMPARE_A = 0;
    OUTPUT_COMPARE_B = 0;
    INTERRUPT_MASK = 0;
    INTERRUPT_FLAGS = 0;
}

/// Configure timer mode which requires writing of 2 separate registers.
static void set_waveform_generation_mode(uint8_t mode) {
     SET_BITS(CONTROL_REGISTER_A, TCCR0A_WGM_MODE_OFFSET, mode & TCCR0A_WGM_MODE_MASK);
     SET_BITS(CONTROL_REGISTER_B, TCCR0B_WGM_MODE_OFFSET, (mode >> 2) & TCCR0B_WGM_MODE_MASK);
}
```

Finally implement the public interface `setup_timer0` which should:

- Reset module
- Set WGM to clear on timer compare (CTC)
- Set prescaler to 1024
- Set output compare register A to the valeu computed for 10 ms cycle
- Set the bit matching the compare A match interrup in the interrupt mask register (TIMSK)
- Reset timer count to 0
- Print out all register values to serial as hexadecimals (`%x`) one on each line to allow diagnosing of configuration issues.

1.8 Analogue-Digital Converter (ADC) driver

Create and add to project the header file for the ADC driver (`adc.h`)

```
#ifndef ADC_H
#define ADC_H
#include <stdint.h>

/** Set up the ADC conversion.
These functions converts input voltage on A3 into 10 bit digital value (0-1023) */
void setup_adc(void);

/**
 * This function is used to poll the latest ADC conversion result.
 * @param  uint16_t *  result   Pointer to location where to store the result of conversion
 *
 * return bool  True if the result was ready and written to value pointed by result,
 * false otherwise
 * */
bool try_reading_adc(uint16_t *result);

#endif //ADC_H
```

Copy public interface definitions to a new file `adc.c` Add it to the project as well.

The ADC driver follows the same structure as the Timer 0 driver. First add includes and define driver-specific constants using the datasheet.

```
#include "adc.h"
#include "critical.h"
#include <avr/interrupt.h>
#include <avr/io.h>
#include <stdint.h>
#include <stdbool.h>
#include "bit_ops.h"

/// Human-readable names
#define ADC_MULTIPLEXER (ADMUX)
#define ADC_CONTROL_STATUS_A (ADCSRA)
#define ADC_CONTROL_STATUS_B (ADCSRB)

/// Datasheet p. 281
#define ADMUX_MUX_OFFSET !!!
#define ADMUX_MUX_MASK !!!

#define ADMUX_MUX_SINGLE_A3 !!!

#define ADMUX_REFERENCE_SELECT_OFFSET !!!
#define ADMUX_REFERENCE_SELECT_MASK !!!

#define ADMUX_REFERENCE_SELECT_AVCC !!!

/// Datasheet p. 285
#define ADCSRA_CLOCK_PRESCALER_OFFSET !!!
#define ADCSRA_CLOCK_PRESCALER_MASK !!!

#define ADCSRA_CLOCK_PRESCALER_DIV_32 !!!

#define ADCSRA_ADC_INTERRUPT_ENABLE_OFFSET !!!
#define ADCSRA_AUTO_TRIGGER_OFFSET !!!
#define ADCSRA_ADC_ENABLE_OFFSET !!!

/// Datasheet p. 287
```

```
#define ADCSRB_TRIGGER_OFFSET !!!
#define ADCSRB_TRIGGER_MASK !!!

#define ADCSRB_TRIGGER_TIMER0_COMPARE_MATCH_A !!!

#define ADCSRB_MUX_OFFSET !!!
#define ADCSRB_MUX_MASK !!!
```

Then implement the ISR and module-global variables that will be used to communicate between the ISR/hardware and the rest of the program.

```
static uint16_t latest_adc_value = 0; ///< to hold the latest ADC conversion result
static bool adc_conv_complete = false; ///< a flag that indicates ADC conversion is completed


/* When ADC conversion is completed, the result will be read and set it to true */
ISR (ADC_vect)
{
    latest_adc_value = ADC;
    adc_conv_complete = true;
}
```

Implement also the public getter `try_reading_adc`. Don't forget to enter critical section before reading variables shared with the ISR and exit it before returning from the call. To simplify reasoningabout the critical section scope keep only a single return statement in this implementation.

```
/**
 * This function is used to poll the latest ADC conversion result.
 * @param  uint16_t *  output   Pointer to location where to store the result of conversion
 *
 * return bool  True if the result was ready and written to value pointed by result, false otherwise
 * */
bool try_reading_adc(uint16_t *output) {
    ...
}
```

Following the patterns of timer0.c implement the rest of ADC initialization. `select_input_channel` is similar to setting waveform generation mode in the timer. `setup_adc` should do the following:

- Reset the module
- Select ADC reference voltage (Avcc)
- Enable internal prescaler (divided by 32) for ADC sampling clock
- Link the ADC auto trigger to Timer/Counter0 compare match A
- Enable ADC interrupt
- Enable Auto Trigger mode
- Enable ADC
- Log values of ADC registers for debugging.

```
/// Write selected multiplexer mode to bits of ADC_MULTIPLEXER and ADC_CONTROL_STATUS_B
static void select_input_channel(uint8_t adc_mux) {
        ...
}

static void reset_adc_module(void) {
   ...
```

```
}

/** Set up the ADC conversion.
 *These functions converts input voltage on A3 into 10 bit digital value (0-1023)
 **/
void setup_adc(void) {
   ...
}
```

1.9 Continuing main.c

Return to main.c and include just created timer0.h, adc.h and the following library headers

```
#include "uart.h"
#include "timer0.h"
#include "adc.h"

#include <util/delay.h>
#include <string.h>
#include <stdio.h>
#include <avr/interrupt.h>
#include <avr/sleep.h>
#include <stdbool.h>
```

1.10 LCD write helper

First, copy LCD write helper from exercise 4 solution.

1.11 Initialization

Write `handle_error` function similar to previous exercises, then write a `void setup(void)` function that implements the following initialization steps.

- Initialize serial port for standard library
- Set Port C pin 3 as input without pull-up.
- Call setup function of the Timer 0
- Call setup function of the ADC
- Configure sleep mode of the processor `set_sleep_mode(SLEEP_MODE_IDLE);`
- Enable interrupts globally with `sei();`
- Initialize LCD by calling `lcd_init(LCD_DISP_ON);`
- Clear LCD screen with `lcd_clrscr();`
- Write 'Ready' to LCD with `write_to_lcd` implemented before
- Busy-wait 1 second

Log progress of initialization to serial port starting right after calling `setup_uart_io();` and checking its exit code.

1.12 Main function and loop

Start the main function by declaring a buffer for formatting strings and memory variable.

```
int main(void) {
        setup();

        uint16_t adc_result = 0;
        uint32_t previous_time = 0;
        uint32_t new_time = 0;

        while (1) {
```

In the while loop:

- Try getting the latest ADC result with `try_reading_adc` interface.
- If value obtained format it to a buffer string with `snprintf` and write to LCD: (line 1) "The ADC result", (line 2) the mearured value.
- Obtain update counter by reading `get_time()` intterface and right shifting it by 6 bits (equivalent to division by 64). This will change value every 10*64 -> 640 ms.
- Loop in while until the value of update counter changes. On every iteration call `sleep_mode()` to cause processor to go into energy-conservation mode. Poll the time after this call to update the exit condition once the CPU is woken up by some interrupt.